

神经网络

Input → 隐藏层 → output P42-P56 网课

↓ 提取一些特征

简易实现线性回归模型 (使用 Pytorch)

- 创建模型: 导入 torch 里面有的 nn (神经网络的包), 定义全连接层, 运用到的 API 有 `nn.Sequential (nn.Linear (input-fe, output-fe))`,
在有多层线性层时, 中间要加上激活函数, 比如在线性回归模型中, 如果只有非线性激活函数, 则网络无法学习复杂的非线性关系, 那模型泛化能力自然不会太好
 - 初始化参数: 通过 `net [n]` 选择初始化的层, 然后用 `weight.data` 和 `bias.data` 访问相应的属性
 - 定义损失函数: `loss = nn.MSELoss()`, 使用 `MSELoss` 类
 - 定义优化算法: 在 `torch.optim` 中有几个优化算法, 其中 SGD 为随机梯度下降算法
完整的 pytorch 实现为: `trainer = torch.optim.SGD (net.parameters(), lr=)`
↓ 指调整的参数 一般为 w 和 b ↓ learning rate 学习率
- 训练: 加入数据向前传播

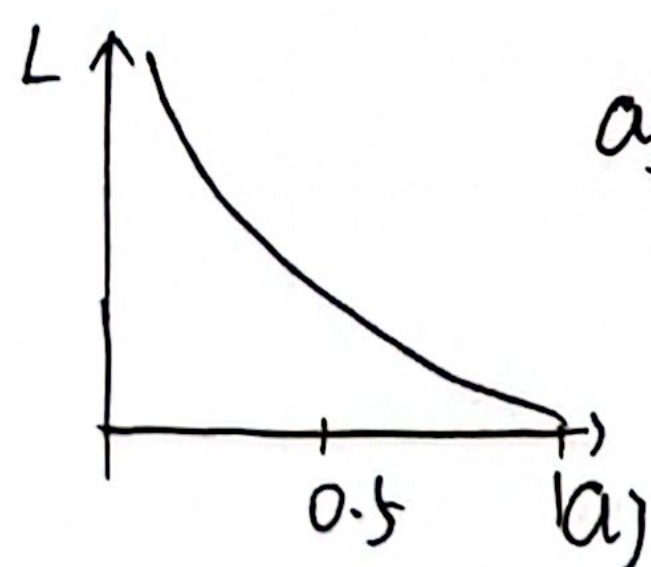
softmax 回归 在分类问题时有多输出的需求 → 当只有两种输出时会变成逻辑回归模型

假定有四种输出, $z_1 = \vec{w}_1 \cdot \vec{x} + b_1 \dots$

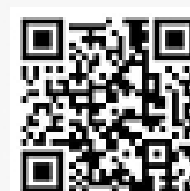
这样求幂的目的是保证非负

$$\text{则 } P(y=i|\vec{x}) = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} = a_i$$

$$\text{loss}(a_1 \dots a_n, y) = -\log a_j \quad y=j$$



$$a_j \rightarrow 1 \quad \text{则} \quad \text{loss} \rightarrow 0$$



CS 扫描全能王

3亿人都在用的扫描App